DATA 260 Homework 6

Rental Housing Listings: MongoDB Memory System

Repository: https://github.com/akankshashuklaWork/data260-7838

Configuration: SID4 7838 | port 8638 | database s7838_doc | prefix s7838 | seed 7838 | verification seed 267838 | Domain 6 - Rental Housing Listings.

Hardware: MacBook Pro, Apple M4 Pro (14-core), 48 GB unified memory. Model: local Ollama qwen2:7b via the existing src/model_client.py adapter. Verification commit hash: 833fabde0c84de0e7f981a3676770b129767839e. Submission tag: hw6.

Part 1 - MongoDB Rental Listings CRUD

This view stores rental listings as MongoDB documents. The validator requires a 1-100 character property title and an availability datetime, adds created/updated timestamps, and limits the status and category values. The FastAPI router supports create, list, get-one, update, and delete operations with JSON CORS and clear validation/error responses.

Code snippet

```
@router.post("", status_code=status.HTTP_201_CREATED) def
create_listing(payload: RentalListingCreate) -> dict:
    document = payload.model_dump(mode="python")
document["listing_status"] = document["listing_status"].value
document["property_category"] = document["property_category"].value
timestamp = utc_now()     document["created_at"] = timestamp
document["updated_at"] = timestamp     result =
listings_collection().insert_one(document)
    return serialize_document(listings_collection().find_one({"_id": result.inserted_id}))
```

Output screenshot

```
$ make verify-hw06
POST /api/rental-listings                       201 Created
GET /api/rental-listings/{listing_id}           200 OK
PUT /api/rental-listings/{listing_id}           200 OK (status: pending)
GET /api/rental-listings                        200 OK
POST /api/rental-listings {}                    422 validation error
GET /api/rental-listings/not-an-id              400 invalid ObjectId
DELETE /api/rental-listings/{listing_id}        204 No Content
```

API smoke-test results. The temporary listing is deleted at the end of the test.

Part 1 - MongoDB Validator and Collection Evidence

The rental_listings validator enforces the required fields, optional description, two enum fields, dates, and timestamps. The initializer also creates the messages, summaries, and episodes collections and their indexes.

Code snippet

```
LISTING_VALIDATOR = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["property_title", "listing_status", "property_category",
                     "available_on", "created_at", "updated_at"],
        "properties": {
            "property_title": {"bsonType": "string", "minLength": 1, "maxLength": 100},
            "property_description": {"bsonType": ["string", "null"], "maxLength": 2000},
            "listing_status": {"enum": ["available", "pending", "rented"]},
            "property_category": {"enum": ["Apartment", "House", "Condo", "Townhouse"]},
            "available_on": {"bsonType": "date"},
        },
    }
}
```

Output screenshot

```
$ mongosh s7838_doc --quiet --eval '<collection snapshot>'
{
  "database": "s7838_doc",
  "collections": [
    "episodes",
    "messages",
    "rental_listings",
    "summaries"
  ],
  "counts": {
    "rental_listings": 0,
    "messages": 282,
    "summaries": 40,
    "episodes": 141
  }
}
```

Live mongosh collection snapshot for s7838_doc. The validator and indexes are in code/hw6_memory/db.py.

Part 2 - Conversation Memory in MongoDB

POST /api/chat saves each user and assistant message, builds a prompt from recent context, summaries, and relevant episodic facts, and returns the reply with memory evidence. GET /api/memory/{user_id} returns the stored messages, summaries, and episodes. GET /api/aggregate/{user_id} returns daily message counts and recent summaries.

Session summaries are created after every five user messages. A lifetime summary is refreshed after every second session summary. In the summary-first setting, a created session summary permits eviction to the four newest prior messages while summaries and episodic facts preserve older needs.

Code snippet

```
user_document = save_message(user_id, session_id, "user", message) short_term = load_short_term(user_id,
session_id, user_document["_id"], window_limit) lifetime_summary = latest_summary(user_id, "user") episodic
= retrieve_episodes(user_id, message, settings.episodic_top_k) prompt, component_tokens =
build_prompt(short_term, session_summary, lifetime_summary, episodic, message) result =
```

```
get_model_client().complete(prompt) save_message(user_id, session_id, "assistant",
result["content"].strip()) stored_episodes = store_episodes(user_id, session_id, message)
generated_session_summary, generated_lifetime_summary = maybe_summarize(user_id, session_id)
```

Output screenshot

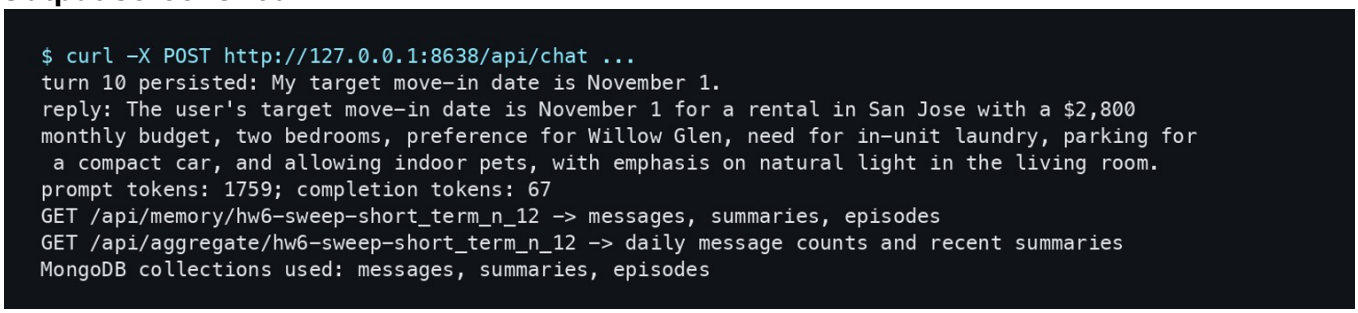


Figure 3. Chat response, token record, memory inspection, and aggregate results from the test run.

Memory/probe evidence

Fixed probe	Expected criterion
Monthly budget	$2,800
Bedrooms	two bedrooms
Neighborhood	Willow Glen
Pet policy	one indoor cat allowed
Move-in	November 1

The full questions, criteria, and 1-5 rubric are stored in reports/hw06/memory_probes.yaml; all runs use the same file.

Part 3 - Token, Latency, and Cost Experiment

I ran 30 fixed conversation messages and five fixed probes in a fresh session for each setting using local Qwen. Each turn logs token estimates, component estimates, latency, policy, context size, and episodic retrieval count. The raw JSON, JSONL, and CSV files are saved in reports/hw06/raw/.

Code snippet

```
telemetry = {     "short_term_n":
settings.short_term_n,
    "memory_policy": settings.memory_policy,
    "prompt_tokens": result["input_tokens"],
    "completion_tokens": result["output_tokens"],
    "component_tokens": component_tokens,
    "latency_ms": latency_ms,
    "episodic_retrieved": len(episodic),
}
_append_telemetry(telemetry)
```

Output screenshot

```
$ make experiment-hw06
short_term_n_4: prompt=1644.63, latency=2317.32 ms, quality=4.60
short_term_n_12: prompt=1908.83, latency=2527.91 ms, quality=5.00
short_term_n_30: prompt=2007.63, latency=2430.65 ms, quality=5.00
summary_first_eviction: prompt=1526.23, latency=1287.85 ms, quality=5.00
Wrote reports/hw06/raw/*.json, per_turn_sweep.csv, and METRICS.md
```

Figure 4. Local Qwen2:7b experiment summary.

Metrics and cost model

Setting	Prompt	Completion	Latency ms	Quality	10K cost
short_term_n_4	1644.63	108.00	2317.32	4.60	$655.39
short_term_n_12	1908.83	82.97	2527.91	5.00	$697.10
short_term_n_30	2007.63	44.53	2430.65	5.00	$669.09
summary_first_eviction	1526.23	47.07	1287.85	5.00	$528.47

Cost assumes 10,000 30-turn conversations at $1 per 1M input tokens and $5 per 1M output tokens. Best: summary_first_eviction (quality 5.00 and lowest tied cost). Worst: short_term_n_12 (highest modeled cost).

Part 4 - File-Based Long-Term Memory

AGENT.md includes project, stack, coding, testing, memory-handling, and prohibited-action instructions. MEMORY.md is loaded into the system prompt. Consolidation runs when the file exceeds 40 non-comment lines or is older than 24 hours; /memory, /compact, /dream, and /help are available.

Code snippet

```
def handle_memory_command(message: str, recent: list[dict], memory_path: Path | None = None) -> str | None:
    command = message.strip().split(maxsplit=1)[0].casefold()
if command == "/memory":
        return load_file_memory(memory_path).content
if command == "/compact":
        result = compact_memory(memory_path)         return f"Compacted MEMORY.md from {result.before_count}
to {result.after_count} non-comment lines."     if command == "/dream":
        return dream_memory(recent)
if command == "/help":
        return "Commands: /memory, /compact, /dream, /help"
return None
```

Output screenshot

```
Akanksha Shukla | HW6 file-memory command demo
Command: /compact
Result: Compacted MEMORY.md from 55 to 39 non-comment lines (17 duplicate facts merged; 3
contradictions resolved explicitly).
```

/compact run using a temporary copy of the original 55-line memory file.

MongoDB conversation memory compared with Markdown file memory

Dimension	MongoDB memory	Markdown memory
Storage	messages, summaries, episodes collections	AGENT.md and MEMORY.md in version control
Trigger	per chat turn; summaries every 5 user messages	>40 non-comment lines or older than 24 hours; /compact
Retrieval	recent window, summaries, cosine-ranked episodes	loaded into system prompt as durable project context
Conflict handling	facts remain reviewable as stored documents	explicit resolved-contradictions section; no silent deletion

Verification and Submission Checklist

I ran the verification script after the implementation was complete. It starts the FastAPI service, checks the configuration, exercises CRUD and error responses, exercises the chat/memory/aggregate endpoints, and tests the 55-line consolidation result. The smoke test uses a deterministic adapter so it does not consume a model call; the recorded experiment uses local Qwen2:7b.

Output screenshot

```
$ make verify-hw06
FastAPI health, configured MongoDB database, CRUD endpoints: PASS
Pydantic validation and invalid ObjectId handling: PASS
chat, memory, aggregate endpoints: PASS
55-line memory consolidation: 55 -> 39; duplicates and contradictions: PASS
AGENT.md instruction check and memory loader check: PASS
all_checks_passed: true
```

Final verification result: all checks passed.

Requirement	Evidence / status
Code under existing root	code/hw6_memory/ and root Makefile; no duplicate repository
Raw experiment evidence	per-setting JSON, per-turn JSONL, memory-probe JSON, eviction comparison JSON, combined CSV

Report and AI record	reports/hw06/report.pdf, Shukla_HW6.pdf, AI_USE.md
Verification	reports/hw06/verification.json reports all_checks_passed: true
Commit and tag	Run git tag hw6 after committing the complete working tree; verification commit hash shown on page 1
GitHub collaborators	Must be confirmed in GitHub repository settings before submission: Sbnikitha and supriyaselvanganesan

AI use is documented in reports/hw06/AI_USE.md with four questions, how each response was used, and source attribution.